

Member visit

Document #: P2637R2
Date: 2023-04-16
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 `std::visit`](#)

[2.2 `std::visit\_format\_arg`](#)

[2.3 Implementation](#)

[3 Wording](#)

[3.1 Feature-test macro](#)

[4 References](#)

§ 1 Revision History

Since [[P2637R1](#)], deprecating `std::visit_format_arg` and thus adding a feature-test macro.

Since [[P2637R0](#)], dropped `apply`, added member `visit<R>` to `basic_format_arg`, and added support for types privately inheriting from `std::variant` for member `visit` and `visit<R>`

§ 2 Introduction

The standard library currently has two free function templates for variant visitation: `std::visit` and `std::visit_format_arg`. The goal of this paper is to add member function versions of each of them, simply for ergonomic reasons. This paper adds no new functionality that did not exist before.

§ 2.1 `std::visit`

`std::visit` is a variadic function template, which is the correct design since binary (and more) visitation is a useful and important piece of functionality. However, the common case is simply unary visitation. Even in that case, however, a non-member function was a superior implementation choice for forwarding const-ness and value category.^{[1](#)}

But this decision logic changes in C++23 with the introduction of deducing `this` [[P0847R7](#)]. Now, it is possible to implement unary `visit` as a member function without any loss of functionality. We simply

gain better syntax:

Existing	Proposed
<pre>std::visit(overload{ [](int i){ std::print("i={}\n", i); }, [](std::string s){ std::print("s= {?:}\n", s); } }, value);</pre>	<pre>value.visit(overload{ [](int i){ std::print("i={}\n", i); }, [](std::string s){ std::print("s= {?:}\n", s); } });</pre>

§ 2.2 std::visit_format_arg

One of the components of the format library is `basic_format_arg<Context>` (see 22.14.8.1 [\[format.arg\]](#)), which is basically a `std::variant`. As such, it also needs to be visited in order to be used. To that end, the library provides:

```
template<class Visitor, class Context>
decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context>
    arg);
```

But here, the only reason `std::visit_format_arg` is a non-member function was to mirror the interface for `std::visit`. There is neither multiple visitation nor forwarding of value category or const-ness here. It could always have been a member function without any loss of functionality. With deducing `this`, it can even be by-value member function.

This example is from the standard itself:

Existing	Proposed
<pre>auto format(S s, format_context& ctx) { int width = visit_format_arg([](auto value) -> int { if constexpr (!is_integral_v<decltype(value)>)) throw format_error("width is not integral"); else if (value < 0 value > numeric_limits<int>::max()) throw format_error("invalid width"); else return value; }, ctx.arg(width_arg_id)); return format_to(ctx.out(), " {0:x<{1}}", s.value, width); }</pre>	<pre>auto format(S s, format_context& ctx) { int width = ctx.arg(width_arg_id).visit([] (auto value) -> int { if constexpr (!is_integral_v<decltype(value)>)) throw format_error("width is not integral"); else if (value < 0 value > numeric_limits<int>::max()) throw format_error("invalid width"); else return value; }); return format_to(ctx.out(), " {0:x<{1}}", s.value, width); }</pre>

The proposed name here is just `visit` (rather than `visit_format_arg`), since as a member function we don't need the longer name for differentiation.

In an [LEWG telecon](#), since adding a member visit to `basic_format_arg` is just strictly better than `std::visit_format_arg`, there's no reason to preserve the existing one (which was only added as a non-member function for consistency with `std::visit`), so LEWG preferred to deprecate `std::visit_format_arg` as part of this paper with a vote of:

SF	F	N	A	SA
1	8	5	1	1

Because `std::visit_format_arg` is being deprecated, now we do require a feature-test macro for this change - which previously wouldn't have needed one.

§ 2.3 Implementation

In each case, the implementation is simple: simply redirect to the corresponding non-member function. Member visit, for instance:

```
template <class... Types>
class variant {
public:
    template <int=0, class Self, class Visitor>
    constexpr auto visit(this Self&& self, Visitor&& vis) -> decltype(auto) {
        return std::visit(std::forward<Visitor>(vis), (copy_cvref_t<Self,
            variant>&&)self);
    }

    template <class R, class Self, class Visitor>
    constexpr auto visit(this Self&& self, Visitor&& vis) -> decltype(auto) {
        return std::visit<R>(std::forward<Visitor>(vis), (copy_cvref_t<Self,
            variant>&&)self);
    }
};
```

`copy_cvref_t<A, B>` is a metafunction that simply pastes the const/ref qualifiers from A onto B. It will be added by [\[P1450R3\]](#).

The C-style cast here is deliberate because `variant` might be a private base of `Self`. This is a case that `std::visit` does not support, but LEWG preferred if member visit did.

There is also an extra leading `int=0` template parameter for the overload that just calls `std::visit` (rather than `std::visit<R>`). This is because, unlike with the non-member functions, an ambiguity would otherwise arise if you attempted to do:

```
using State = std::variant<A, B, C>;

State state = /* ... */;
state = std::move(state).visit<State>(f);
```

With non-member visit, there's no real possible ambiguity because the function goes first. But here, unless we protect the non-return-type taking overload, `Self` could deduce as `State`, which would then be a perfectly valid overload. And this pattern isn't rare either - so it's important to support. The added `int=0`

parameter ensures that only the first overload is viable for `v.visit(f)` and only the second is viable for `v.visit<R>(f)`.

§ 3 Wording

Add to 22.6.3.1 [\[variant.variant.general\]](#):

```
namespace std {
    template<class... Types>
    class variant {
    public:
        // ...

        // [variant.status], value status
        constexpr bool valueless_by_exception() const noexcept;
        constexpr size_t index() const noexcept;

        // [variant.swap], swap
        constexpr void swap(variant&) noexcept(see below);

+     // [variant.visit], visitation
+     template<class Self, class Visitor>
+         constexpr see below visit(this Self&&, Visitor&&);
+     template<class R, class Self, class Visitor>
+         constexpr R visit(this Self&&, Visitor&&);
    };
}
```

Add to 22.6.7 [\[variant.visit\]](#), after the definition of non-member visit:

```
template<class Self, class Visitor>
    constexpr see below visit(this Self&& self, Visitor&& vis);

9  Let V be OVERRIDE_REF(Self&&, COPY_CONST(remove_reference_t<Self>, variant))
    ([forward]).

10 Constraints: The call to visit does not use an explicit template-argument-list that begins
    with a type template-argument.

11 Effects: Equivalent to return std::visit(std::forward<Visitor>(vis), (V)self);

    template<class R, class Self, class Visitor>
        constexpr R visit(this Self&& self, Visitor&& vis);

12 Let V be OVERRIDE_REF(Self&&, COPY_CONST(remove_reference_t<Self>, variant))
    ([forward]).

13 Effects: Equivalent to return std::visit<R>(std::forward<Visitor>(vis), (V)self);
```

Remove from 22.14.1 [\[format.syn\]](#) (this is now deprecated):

```

namespace std {
    // ...
    // [format.arguments], arguments
    // [format.arg], class template basic_format_arg
    template<class Context> class basic_format_arg;

-   template<class Visitor, class Context>
-   decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context>
        arg);
    // ...
}

```

Change the example in 22.14.6.6 [\[format.context\]/8](#):

```

struct S { int value; };

template<> struct std::formatter<S> {
    size_t width_arg_id = 0;

    // Parses a width argument id in the format { digit }.
    constexpr auto parse(format_parse_context& ctx) {
        auto iter = ctx.begin();
        auto get_char = [&]() { return iter != ctx.end() ? *iter : 0; };
        if (get_char() != '{')
            return iter;
        ++iter;
        char c = get_char();
        if (!isdigit(c) || (++iter, get_char()) != '}')
            throw format_error("invalid format");
        width_arg_id = c - '0';
        ctx.check_arg_id(width_arg_id);
        return ++iter;
    }

    // Formats an S with width given by the argument width_arg_id.
    auto format(S s, format_context& ctx) {
-       int width = visit_format_arg([](auto value) -> int {
+       int width = ctx.arg(width_arg_id).visit([](auto value) -> int {
            if constexpr (!is_integral_v<decltype(value)>)
                throw format_error("width is not integral");
            else if (value < 0 || value > numeric_limits<int>::max())
                throw format_error("invalid width");
            else
                return value;
-       }, ctx.arg(width_arg_id));
+       });
        return format_to(ctx.out(), "{0:x<{1}}", s.value, width);
    }
};

std::string s = std::format("{0:{1}}", S{42}, 10); // value of s is
"xxxxxxxx42"

```

Add to 22.14.8.1 [\[format.arg\]](#):

```

namespace std {
    template<class Context>
    class basic_format_arg {
        // ...
    public:
        basic_format_arg() noexcept;

        explicit operator bool() const noexcept;

+   template<class Visitor>
+   decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);
+   template<class R, class Visitor>
+   R visit(this basic_format_arg arg, Visitor&& vis);

    };
}

```

And:

```
explicit operator bool() const noexcept;
```

7 *Returns:* `!holds_alternative<monostate>(value)`.

```

template<class Visitor>
    decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);

```

8 *Effects:* Equivalent to `return arg.value.visit(std::forward<Visitor>(vis));`

```

template<class R, class Visitor>
    R visit(this basic_format_arg arg, Visitor&& vis);

```

9 *Effects:* Equivalent to `return arg.value.visit<R>(std::forward<Visitor>(vis));`

And remove 22.14.8.1 [\[format.arg\]](#)/13:

```

template<class Visitor, class Context>
decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context>
arg);

```

13 ~~*Effects:* Equivalent to `return visit(std::forward<Visitor>(vis), arg.value);`~~

Add a new clause in D [\[depr\]](#): [depr.format]:

1 The header `<format>` has the following addition:

```

namespace std {
    template<class Visitor, class Context>
        decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context>
            arg);
}

template<class Visitor, class Context>
    decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context>
        arg);

```

2 *Effects:* Equivalent to `return visit(std::forward<Visitor>(vis), arg.value);`

§ 3.1 Feature-test macro

Bump both `__cpp_lib_variant` and `__cpp_lib_format` in 17.3.2 [\[version.syn\]](#)

```
- #define __cpp_lib_format 202207L // also in <format>
+ #define __cpp_lib_format 2023XXL // also in <format>

- #define __cpp_lib_variant 202106L // also in <variant>
+ #define __cpp_lib_variant 2023XXL // also in <variant>
```

§ 4 References

[P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this.
<https://wg21.link/p0847r7>

[P1450R3] Vincent Reverdy. 2020-06-15. Enriching type modification traits.
<https://wg21.link/p1450r3>

[P2637R0] Barry Revzin. 2022-09-17. Member visit and apply.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2637r0.html>

[P2637R1] Barry Revzin. 2022-10-06. Member visit.
<https://wg21.link/p2637r1>

1. A single non-member function template is still superior to four member function overloads due to proper handling of certain edge cases. See the section on [SFINAE-friendly](#) for more information.↩